Theseus- Canada

[St Andrew's College — RoboCup Junior Rescue Maze

TEAM MEMBERS & SOCIALS

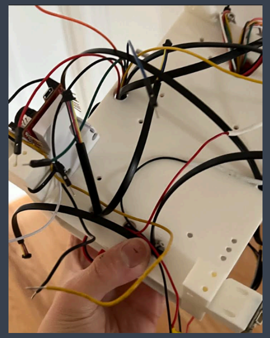
- Baichen Luo - Team Lead (Electronics & Computer Vision)
- Leo Tiralongo - Mechanical Design & Electrical Assembly
- Christopher Shu - Software Architecture & Navigation
- Lucas Cai - BFS Algorithm Programmer

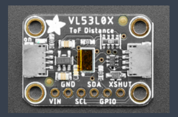
Documentation link: https://github.com/Arsur24/Theseus/tree/main
Code link: https://github.com/Chrisyyyys/Robocup-Junoir-Theseus

INTRODUCTION

What sets the Theseus Maze Robot apart from competitors utilizes a fusion of information from disparate sensors to accurately and reliably traverse the maze. Instead of using a single type of sensor to control movement, data from the gyro, TOF and magnetic encoder sensors are combined to allow for smooth and precise movement through the maze. The robot uses a state machine as its software architecture for navigating the maze, and uses a 2-D Array consisting of our own custom tile bitset to map out its route. To detect victims, a powerful FOMO network was trained on over a thousand augmented images through an edge impulse pipeline to detect victims. We also have many hardware innovations, including a suspension system for smoother movement in uneven terrain as well as a rotating dispenser allowing for selective dispensing on both sides of the robot. However, our team has placed most of our efforts into improving the software, as we believe that better navigation, movement and detection would give us the advantage over a team with custom hardware, but with worse code.

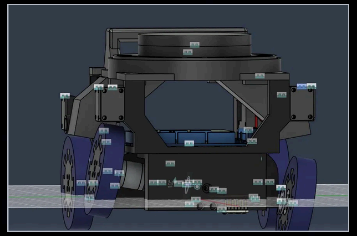
DISTANCE SENSING

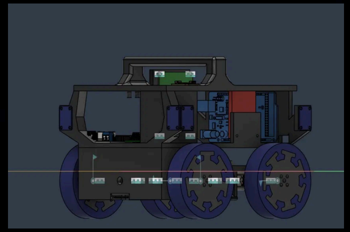




The robot uses an array of VL53L0X time-of-flight distance sensors. These sensors emit infrared light and estimate distance from the light's return time. They are preferable to basic reflective infrared sensors because their measurements are less dependent on the visible colour of the wall.
The README identifies the VL53L0X as the principal distance sensor and emphasizes the organizational advantages of using I²C devices.
The sensors are positioned around the chassis to provide forward obstacle detection, wall measurement in all 4 cardinal directions, and to keep the robot centered in the maze.
Because multiple VL53L0X modules normally start with the same I²C address, they are connected through a TCA9548A/Qwiic I²C multiplexer. The multiplexer electrically separates the sensors into independent channels. The controller selects one channel, reads that sensor, and then switches to the next channel.

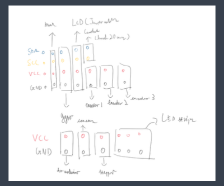
CHASSIS & MATERIALS

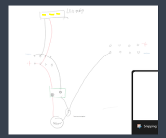




The chassis is 3D printed in PLA for its light weight and ease of production. The robot follows a layered architecture, with each layer serving a dedicated purpose. In V1, the motors were positioned on the base plate, which limited the available space and caused wiring issues that made fitting the top plate difficult. In V2, the motors were moved beneath the base plate, which now houses the dropper mechanism. This freed up the mid layer significantly, allowing for cleaner wiring and better organization of electronics. The suspension uses a dead axle design, where a fixed shaft supports the front wheels through press-fit bearings, preventing torsional stress under repeated use. Cable strain relief posts with capped ends are built into the chassis to prevent tension on motor connections.

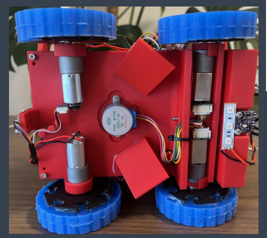
POWER SYSTEM

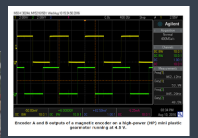


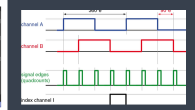


The power system is divided into two branches: a high-current motor rail and a regulated electronics rail. The motor rail powers the drive motors through the Carobot V3 motor shield, while the dispensing stepper motor is supplied via a 5V buck converter. The OpenMV cameras are powered from a dedicated regulated supply to prevent voltage transients from causing camera resets. The regulated logic rail supplies the distance sensors, IMU, and colour sensor. The Arduino GIGA R1 operates on 3.3V logic, so all high-current devices are driven through dedicated driver electronics rather than directly from GPIO pins.

DRIVE SYSTEM / MOTORS

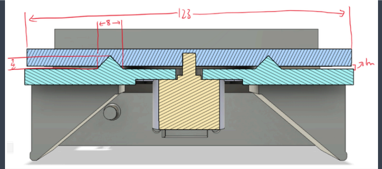






We decided to use 195:1 polulu 12v gearmotors for their high torque, which allows the robot to easily travers obstacles such as ramps and steps.The Polulu encoders(https://www.pololu.com/product/1523) produce two alternating square waves, A and B, when overlapped creates 20 pulses per rotation. In our code, we are only measuring when pin A rises, so thats 20/2(the number of waves A produces per rotation) divided by 2(only the rising).

RESCUE KIT DEPLOYMENT MECHANISM



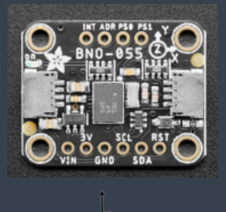
The rotating turntable was selected because it can deploy kits from either side of the robot. This provides a competitive advantage because the robot does not need to rotate 180 degrees before deploying a kit.

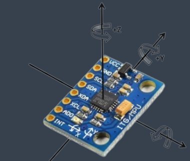
Earlier dispenser designs had several problems. Mounting the motor inside the mechanism wasted space, the guide failed to engage because it had insufficient clearance, and the original high chutes caused rescue kits to bounce away from the target.

The final design uses a stepper motor mounted beneath the dispenser with a press-fit connection to the turntable. The turntable has 19 mm by 19 mm openings for the 10 mm rescue kits. Approximately 1 mm of clearance prevents binding, while 8 mm by 4 mm triangular guides direct kits into the chutes. Two steep chutes extend beyond the wheels, helping kits land within the required 150 mm scoring area.

The stepper motor rotates the turntable in approximately 22-degree increments. The control system selects the left or right chute based on the detected victim, connecting the sensing, software, and mechanical systems.

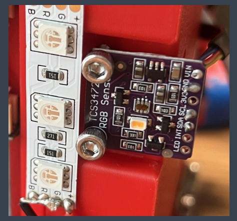
ORIENTATION (IMU)





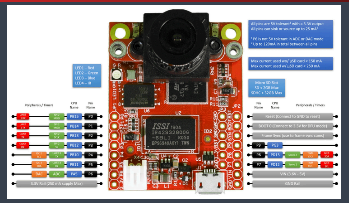
The robot uses an Adafruit BNO055 nine-degree-of-freedom IMU(picture below) for heading measurement
.
An earlier version used the MPU6050(Picture above). However, the MPU6050 primarily provides raw acceleration and angular-velocity data. Determining heading requires integrating the angular velocity over time, which accumulates error and produces drift. The BNO055 performs onboard sensor fusion and can directly report orientation.

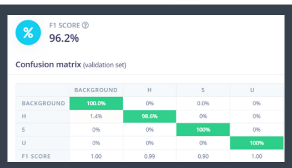
COLOR SENSING



A downward-facing TCS34725 colour sensor inspects the floor beneath the robot via I2C. It provides red, green, blue, and clear-light measurements. The clear-light channel is particularly useful for detecting low-reflectance black hazard tiles, while RGB ratios distinguish coloured tiles from simple reductions in overall brightness. The sensor also detects blue obstacle tiles, triggering a mandatory 5-second stop as required by the rules.

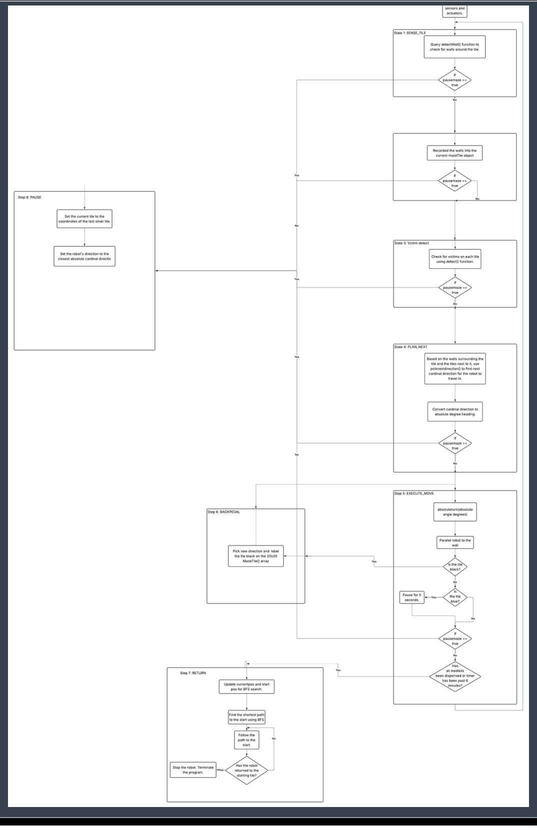
CAMERA & VISION

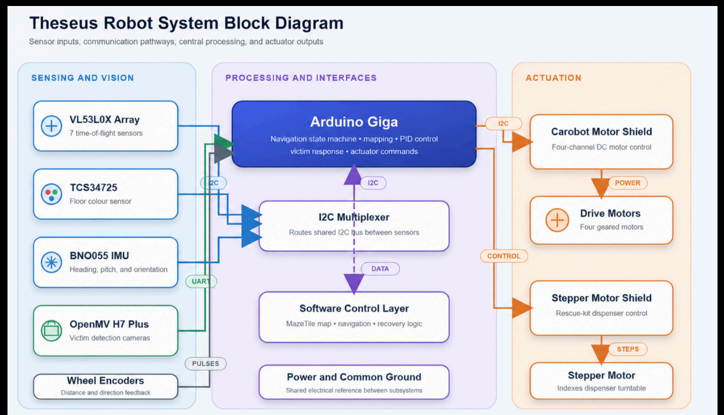


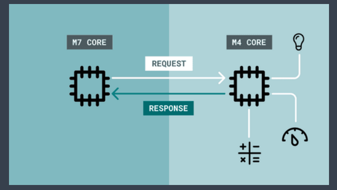


To detect the lettered victims, we use a Edge Impulse pipeline to train a FOMO detector to recognize victims(More information on the model could be found here). One major advantage of FOMO is that it uses x30 less processing power compared to similar models, such as Yolo v5. FOMO works by creating a probabilistic heat map of the image at a certain convolution layer, which is around x8 smaller than the original image. Based on the probability of each pixel, the model is able to find the location of victim.
 A total of 450 images, with 150 images of each letter, was taken on the K210 camera. Before training, the images were preprocessed, first by binarizing them, and after Baichen wrote a preprocessing script on google Collab that produced a rotated and scaled version of each of the images, tripling the dataset to over 1300 images. The model was trained on a batch size of 32, with a scheduled learning rate that has a maximum of 0.001 over 40 epochs. After training, the model was found to have a 96.2% accuracy, which is very high. The confusion matrix could be found below:

SOFTWARE FLOWCHART

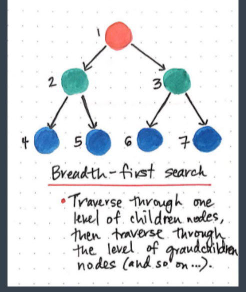






The robot uses a state machine as its core software architecture, navigating tile by tile through the maze. The maze is mapped using a 2D array of custom tile bitsets, storing wall data, victim records, and exploration states. Breadth-first search calculates the return path to the starting tile. The Arduino GIGA's dual-core processor divides responsibilities: the M7 core handles navigation, PID motor control, mapping, and BFS, while the M4 core continuously monitors the OpenMV cameras for victim detections.

SOFTWARE & NAVIGATION

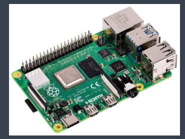


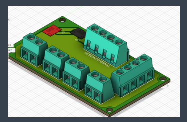
- The robot software uses a hybrid architecture combining high-level maze navigation with reactive movement correction. High-level navigation determines which tile the robot should visit, records the maze, and calculates a return route, and low-level reactive systems operate during movement to maintain alignment, avoid collisions, detect victims, identify hazardous tiles, and respond to ramps.
- The program uses a tile-based state machine. The maze is divided into 300 mm tiles, and the robot completes a structured sequence whenever it reaches a new tile. It senses its surroundings, updates its internal map, selects a direction, turns and aligns itself, detects victims, moves one tile, confirms the result, and repeats. This structure helps keep the robot's physical location synchronized with its stored map.
-

RESULTS & ACHIEVEMENTS

In our first friendly competition on February the 28th, our robot performed much better than in previous years, achieving a score of between 110-200 with detection alone(the dispenser system was not attached yet). In our qualifier run, the robot did slightly worse(95-130 points) due to an unforeseen logic error, and the robot running out of power midway throughout the run. However, in a testing run, the robot was able to detect victims reliably, dispense rescue kits and return to its starting point. We were able to achieve first place in the York regional Robocup Junior competition.

FUTURE IMPROVEMENTS





In future versions, we plan to use a Raspberry Pi to provide additional processing power for advanced vision models such as YOLOv11 and potentially integrate LiDAR-based SLAM for improved localization and mapping. We would also like to increase modularity by designing a custom PCB to simplify wiring and sensor integration. Finally, we are interested in developing a tracked drivetrain, which could improve performance on ramps and steps while providing an exciting engineering challenge in both mechanical design and control.